QUIVER

a quiver of vectors

Quiver, Explained

From "what is a vector?" to a production vector database — a complete, plain-English tour of how a secure, memory-frugal ANN engine actually works.

VECTOR SEARCH · RUST ENGINE · SECURITY-FIRST · MEMORY-FRUGAL

`quiver@cockpit:~$ ./read --audience=everyone --depth=full`

The security-first vector database

Open-source · AGPL-3.0 · single static binary

Authored for engineers & the curious

github.com/achref-soua/quiver

00 FRONT MATTER

How to read this guide & what's inside

One document, written for two readers at once — the newcomer and the engineer.

This guide explains a real, production-grade vector database — **Quiver** — from absolute first principles all the way down to its on-disk format and cryptography. It is deliberately written so that two very different readers get value from the *same* pages.

> TWO READING PATHS
>
> **If you have never touched AI or databases:** read straight through. Every concept is introduced with a real-world analogy and a concrete example before any jargon appears. You can skip the dashed "Under the hood" boxes and still understand everything.
>
> **If you build software:** the same sections carry the depth — algorithms, data structures, trade-offs, and the real numbers, with the design *decisions* explained, not just stated. The dashed boxes go all the way down.

We keep one running example throughout: a **movie-recommendation search** — "find me films that *feel like* Blade Runner." By the end you will know precisely what happens, step by step, when that query runs.

Contents

PART 1 FOUNDATIONS

The Big Idea: Turning Meaning into Numbers

Before any database, one question: how do you make a computer understand that two different things mean *the same thing?*

1.1 The problem with keywords

Imagine you run a movie site. A user types: *"a moody sci-fi about androids and identity."* A traditional database searches with **keywords** — it looks for the literal words "moody," "sci-fi," "androids." But Blade Runner's description might say "a replicant hunter in a neon dystopia." Zero shared keywords — yet it is a perfect match. Keyword search is **blind to meaning**.

What we actually want is **search by meaning, not by spelling**. That is the entire reason vector databases exist.

1.2 What is an "embedding"? *— the single most important idea*

> THE CORE IDEA
>
> You can turn any piece of content — a sentence, an image, a song, a product — into a list of numbers that captures its *meaning*. That list of numbers is called an **embedding** (or a **vector**).

A vector is just an ordered list of numbers, for example:

```
EMBEDDING
# a movie, as a vector of ~768 numbers
Blade Runner  →  [ 0.91, -0.20, 0.74, 0.05, 0.61, -0.42, … ]
```

Each number is a coordinate along some invisible "axis of meaning." Loosely, you can think of these axes as *"how sci-fi is it? how romantic? how violent? how hopeful?"* — except a real AI model discovers thousands of subtle axes on its own, far richer than words we would name.

The magic property: things that *mean* similar things get *nearby* numbers. Blade Runner and Ghost in the Shell end up close together; Blade Runner and Paddington 2 end up far apart. Meaning becomes **geometry**.

THE MENTAL MODEL

Picture a giant map. Every movie is a pin. Similar movies cluster into neighbourhoods — the "gritty sci-fi" district, the "feel-good comedy" district. Searching "find films like Blade Runner" becomes: **drop a pin where the query lands, and look at its nearest neighbours.** A real map has 2 dimensions; an embedding map has hundreds or thousands. The idea is identical.

Who makes these numbers? An **embedding model** — a neural network like OpenAI's `text-embedding-3`, Cohere, or an open model. You feed it text or an image; it returns the vector.

⌥ UNDER THE HOOD

Quiver is deliberately **model-agnostic**: *you* produce the embeddings with whatever model you like, and Quiver stores and searches them. It never bundles a model — a conscious choice, because embedding models change monthly and tying a database to one would age it instantly. Quiver's job starts the moment you have vectors.

1.3 "Similar" means "close" — but close *how*?

If meaning is geometry, then **similarity is distance**. Two questions follow: how do we measure distance, and which measure is right? There are three standard answers, and Quiver supports all three.

FIG. 1 — TWO WAYS TO MEASURE "CLOSE"

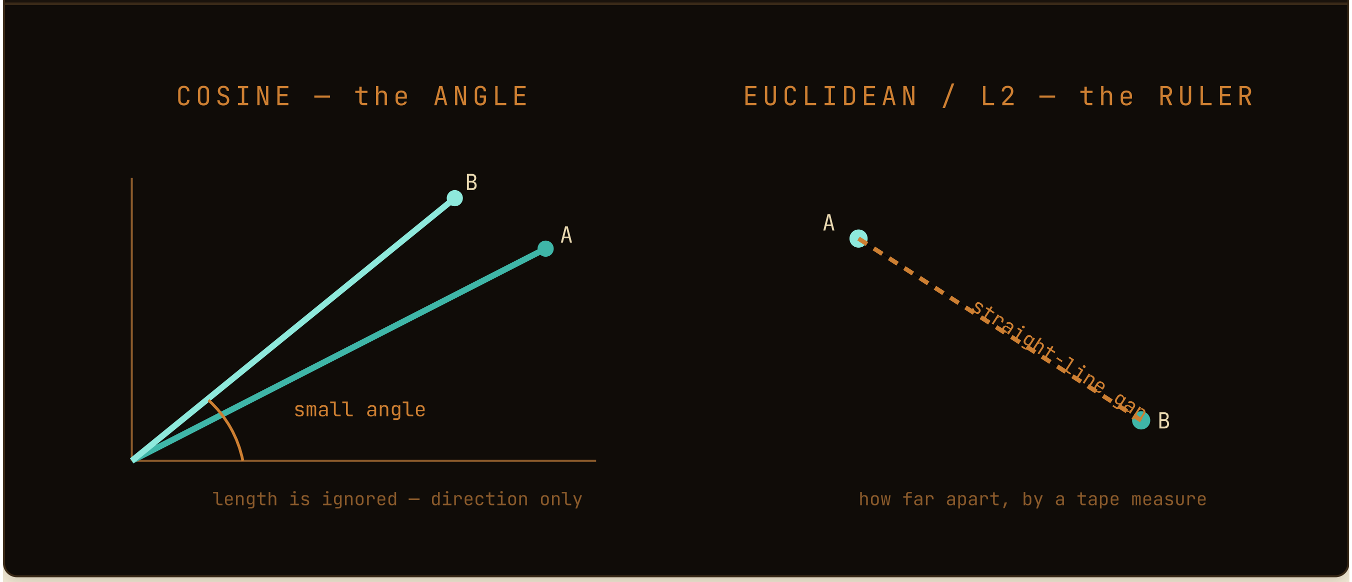

FIG. 1 — Cosine asks "do the arrows point the same way?"; L2 asks "how far apart are the points?".

Table 1 — the three similarity metrics Quiver supports

METRIC	PLAIN-ENGLISH MEANING	BEST FOR	"CLOSER" MEANS
Cosine	Do the two arrows point the *same direction*? (ignores length)	Text / semantic search — the common default	Higher (max 1.0)
Euclidean / L2	How far apart are the points, by a ruler?	Image embeddings, spatial data	Smaller distance
Dot product	Direction *and* magnitude together	Recommenders, "maximum inner product"	Higher

WHY COSINE WINS FOR TEXT

A long document and a short tweet about the same topic point the *same direction* but have very different *lengths*. Cosine ignores length and asks only "same topic?" — exactly what you want. That is why it is the default for semantic search.

⌥ UNDER THE HOOD

Quiver normalises cosine internally: it scales every vector to length 1, after which **cosine reduces to a plain dot product**. So the engine only needs fast code for two operations (dot product and squared-L2) and cosine comes for free. Internally every metric is converted to a uniform *"smaller is closer"* orientation (similarities are negated), so all the search machinery uses one comparison rule and never confuses direction.

1.4 The scaling wall: why we can't just compare everything

Naively, finding nearest neighbours is easy: compare the query to *every* stored vector, sort, take the top few. This is **brute force** (also "exact" or "flat" search). It is also a trap. With 1,000 movies it is instant. With **100 million** documents of 768 numbers each, every single query does ~76 *billion* multiplications. Per query. Hopeless for a live website.

So vector databases make a bargain:

APPROXIMATE NEAREST NEIGHBOUR (ANN)

Give up *guaranteeing* the perfect top-10, in exchange for being **100–1000× faster** while still being right ~95–99% of the time.

That trade — a tiny, controllable sacrifice in accuracy for an enormous speed-up — is the heart of every vector database. The accuracy you keep is called **recall**, and it is the number to watch.

RECALL, DEFINED ONCE

Recall@10 = "of the 10 truly nearest items, how many did the fast approximate search actually find?" Recall of 0.96 means it found 9.6 of the true top 10 on average. Brute force always has recall 1.0 but is slow; ANN trades a sliver of recall for speed. **Every benchmark in this guide is reported at a fixed recall**, because speed without recall is meaningless — you can be infinitely fast by returning garbage.

PART 2 POSITIONING

What Quiver Is, and Why It's Different

Now that the concepts are clear, here is the product — and the narrow, deliberate edge it competes on.

Quiver is an open-source vector database written in Rust. You give it vectors (plus optional metadata like `{"year":1982,"genre":"sci-fi"}`), and it answers "find the *k* most similar" queries in milliseconds — over a network API, an embeddable in-process library, or as a tool that AI agents can call directly.

There are already excellent vector databases — Pinecone, Milvus, Qdrant, Weaviate, FAISS, pgvector. Quiver does not try to beat them at raw scale. It competes on a **narrow, deliberate edge** — three things, executed well.

2.1 The wedge

1 SECURITY-FIRST, BY DEFAULT
Encryption is on out of the box — every durable byte sealed on disk. You can even encrypt vectors so the server itself never sees them. Only audited, industry-standard cryptography.

2 MEMORY FRUGALITY
Serve hundreds of millions of vectors from a laptop's RAM budget, using disk-resident indexes plus compression (~32× less memory). The headline metric is RAM-at-a-fixed-recall.

3 DEVELOPER EXPERIENCE
A single static binary. Embedded or server mode. A retro terminal "cockpit." Python & TypeScript SDKs. An MCP server so AI agents can drive it.

FIG. 2 — Quiver's defensible edge is the **combination** of three things, each executed well.

2.2 And — just as important — what Quiver honestly says it does *not* do

A trustworthy system is clear about its limits. Quiver states plainly:

- It is **single-node first**. It will not out-scale a distributed Milvus cluster. (Async read-replicas exist as a clearly-labelled stretch feature.)

- Client-side encryption protects **payloads, not vectors** — unless you opt into a special experimental mode (Part 6), with documented leakage.
- There is **no fully homomorphic "search on encrypted data with zero leakage"** — because no such scheme is fast enough to be practical today, and it will not pretend otherwise.

WHY THIS HONESTY MATTERS

A database is infrastructure you bet your data on. The project's README carries a striking rule: *"Every performance/memory claim is backed by a reproducible benchmark on documented hardware — until those numbers are recorded, the table stays empty rather than guess."* That discipline — never fabricating a number — is itself a feature.

PART 3 ARCHITECTURE

The Architecture, Top to Bottom

A large codebase stays understandable only when each piece has one job and the dependencies point one way.

3.1 The shape of the system

Quiver is a **Cargo workspace**: a collection of small, focused Rust libraries ("crates"), each with one job, stacked so the low-level engine pieces never depend on the high-level network pieces. The rule the arrows enforce is that **the engine knows nothing about HTTP, gRPC, or terminals** — the network code is a thin shell around the same engine the embeddable library exposes.

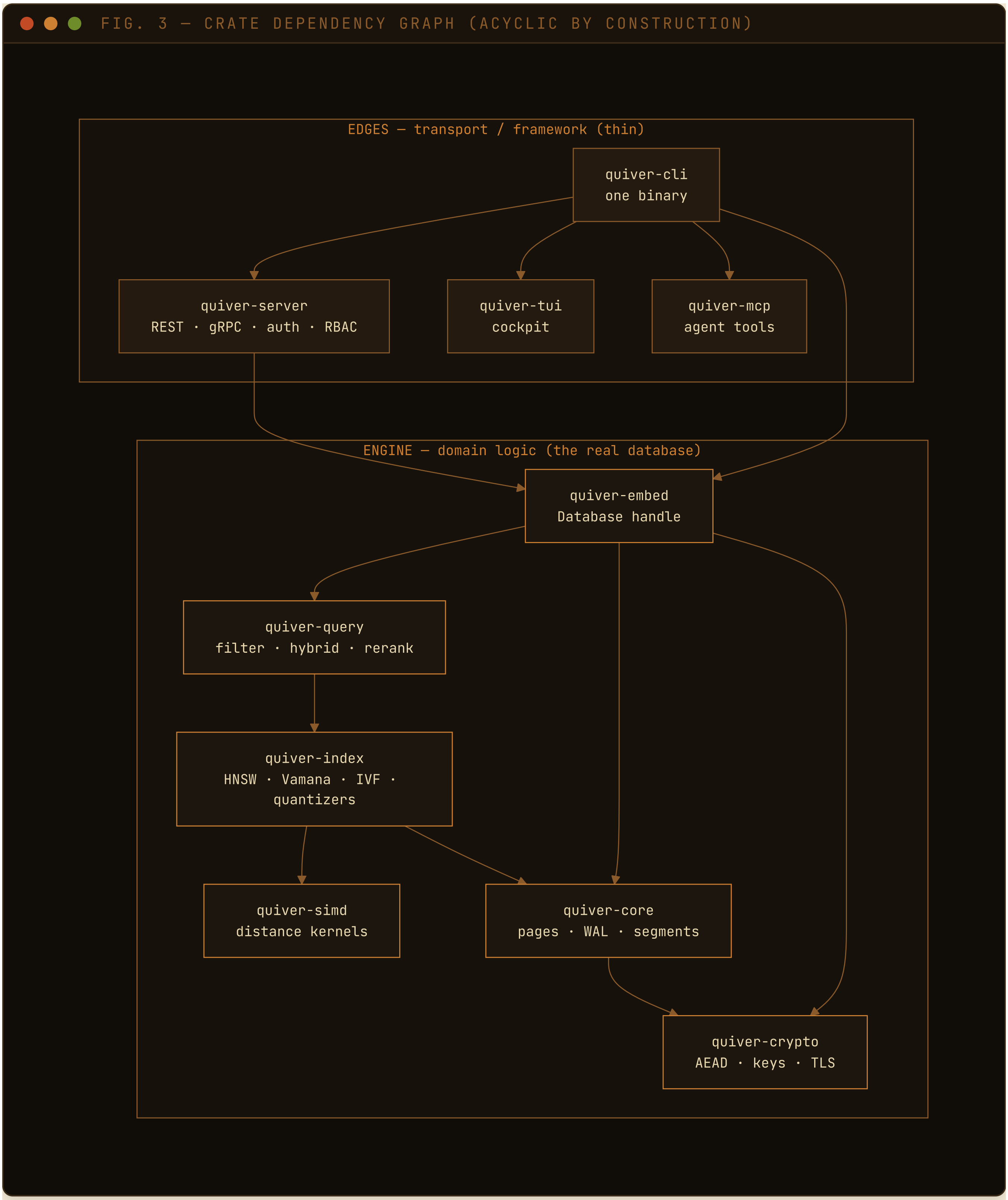

FIG. 3 — The engine (bottom) is reusable and testable in isolation; framework code stays at the edges.

⌥ UNDER THE HOOD — THE CRATE MAP

quiver-simd (~640 lines) — raw arithmetic: distance between two vectors, as fast as the CPU allows. Pure compute, no I/O. **quiver-crypto** (~2,650) — thin, careful wrappers over *audited* cryptography; never a home-grown cipher. **quiver-core** (~6,200) — the storage engine built from scratch: pages, the write-ahead log, segments, the catalog, compaction. *No embedded database (no RocksDB/SQLite/LMDB) is used.* **quiver-index** (~5,700) — the ANN indexes and the quantizers. **quiver-query** (~490) — the query planner: filtering, hybrid search, merge & re-rank. **quiver-embed** (~3,400) — stitches it into one clean `Database` API. The rest are edges. The whole thing is **Rust (edition 2024)**, AGPL-3.0, shipping as **one static binary**; a workspace lint policy even *forbids* `unwrap()` / `expect()`, forcing every error to be handled.

3.2 Two ways to run it

Table 2 — embedded library vs server

MODE	WHAT IT IS	WHEN TO USE
Embedded library	`Database::open(path)` — an in-process handle. No network, no auth surface, but encryption-at-rest still on.	Tests, notebooks, desktop apps, anything local.
Server	`quiver serve` — gRPC + REST with authentication, role-based access, multi-tenant namespaces, audit logging, query cost limits.	Production, shared services.

The same binary does both. The terminal cockpit (`quiver tui`) and the AI-agent server (`quiver mcp`) are just **clients** of the server API, so they work against a local *or* a remote Quiver.

3.3 What happens when you write, and when you search

Let us trace our movie example end to end. First, **writing a movie** ("upsert"). The crucial line is *append* → *fsync* → *durable*: the write is acknowledged only *after* it is flushed to disk.

FIG. 4 — THE WRITE (UPSERT) PATH

1	`Client → Server`	upsert **"Blade Runner"** with its vector and `{year:1982}`
2	`Server`	terminate TLS · authenticate · check role & scope · cost-limit
3	`Server → Engine`	hand off the validated request
4	`Engine → WAL (disk)`	DURABILITY BOUNDARY ✓ append the encrypted, checksummed record, then **fsync** — from here the write survives `kill -9` and power loss
5	`Engine → Index`	insert the vector into the live index
6	`Engine → Client`	acknowledged — the client hears "ok" only now

FIG. 4 — The write is acknowledged only **after** the record is physically on disk.

Now **searching** — "films like Blade Runner, but only sci-fi after 1980." Notice the two-phase pattern that recurs *everywhere* in Quiver: a fast approximate pass to get a shortlist, then a slow exact pass on just that shortlist.

FIG. 5 — THE SEARCH (TOP-K + FILTER) PATH

1	`Client → Planner`	search(query vector, k=10, filter: `genre=sci-fi AND year>1980`)
2	`Query planner`	choose the filter strategy — **pre-filter** if selective, else **post-filter**
3	`Planner → Index`	FAST · APPROXIMATE ANN search over the compressed (quantized) vectors
4	`Index → Storage`	fetch the full-precision vectors for the shortlist only
5	`Query planner`	EXACT · PRECISE ✓ re-rank the shortlist with **exact** distances and re-check the filter on every survivor
6	`Planner → Client`	true top-10 with payloads + a cursor for pagination

FIG. 5 — The two-phase pattern: cheap approximate narrowing, then exact precision on the shortlist.

PART 4 THE ENGINE

The Engine, Block by Block

From the bottom — raw arithmetic — up to the clever data structures that make search fast.

4.1 The speed floor: SIMD distance kernels

Every search, no matter how clever, eventually computes "how far apart are these two vectors?" thousands of times. If that one operation is slow, *everything* is slow. The naïve way multiplies element by element, 768 times. The fast way — **SIMD** ("Single Instruction, Multiple Data") — multiplies *eight numbers at once* with a single CPU instruction. It is the difference between a cashier scanning one item at a time and a scanner that reads eight barcodes in one pass.

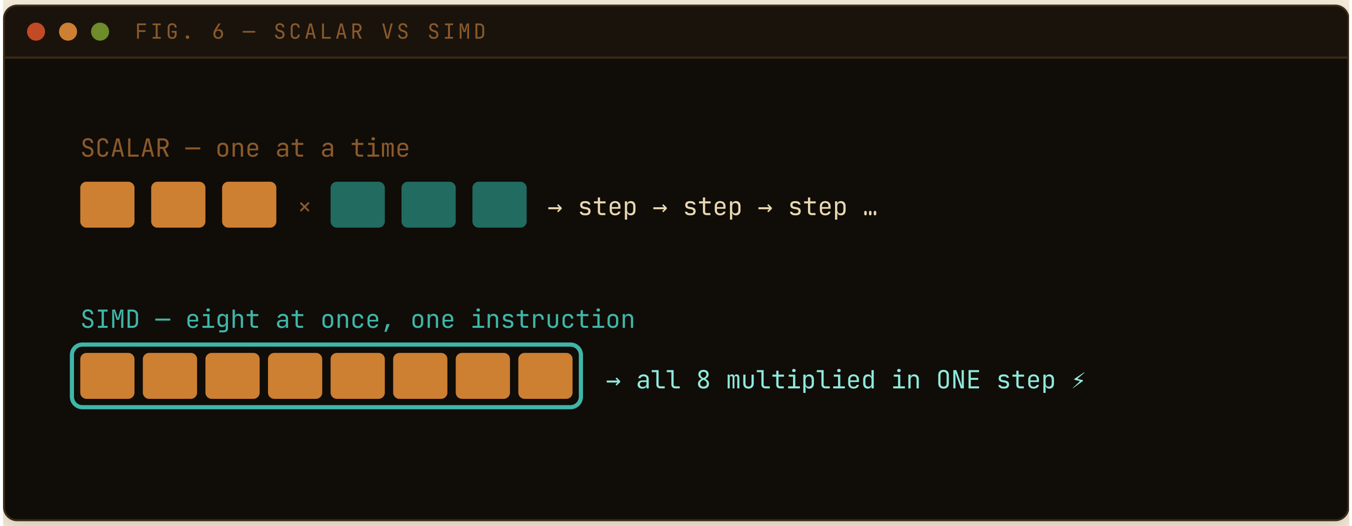

FIG. 6 — The foundational speed-up every index is built on top of.

> ⌥ UNDER THE HOOD
>
> `quiver-simd` provides hand-written kernels for cosine, squared-L2, dot product (over 32-bit floats *and* 8-bit integers), and **Hamming distance** (bit-counting, for binary compression — §4.3). At runtime it *detects the CPU's features* and dispatches to the AVX2 / AVX+FMA path if present, falling back to portable scalar code otherwise — so one binary runs fast on a modern server and *correctly* on an old one. Every SIMD path is **differential-tested**: random vectors of awkward lengths (0, 1, 7, 769 — deliberately not multiples of 8, to exercise the leftover "tail") are fed to both versions and asserted equal. Fast code you cannot trust is worthless; this is how it earns the trust.

4.2 The core trick of ANN: navigable graphs

How do you find nearest neighbours among 100 million vectors *without* comparing them all? The most successful answer is a **proximity graph**.

THE IDEA

Build a network where each vector is a node, connected by "friendship" links to a handful of nearby vectors. To search, **start anywhere and keep walking to whichever friend is closer to your query, until you can't get closer.** Like finding a house by repeatedly asking "which of your neighbours lives closest to this address?" — you converge in a few hops, never visiting the whole city.

Quiver implements the two best-known proximity-graph families: **HNSW** and **Vamana** (**DiskANN**), plus the cluster-based **IVF**.

HNSW — the "skip-list of maps"

HNSW (Hierarchical Navigable Small World) adds one idea to the proximity graph: layers, like a zoomed-out highway map laid over a detailed street map. You enter at the sparse top, take big leaps into the right region, then drop down layer by layer, refining, until the dense bottom layer where every vector lives. Coarse-to-fine, geographically.

FIG. 7 — HNSW: SEARCH DESCENDS FROM SPARSE HIGHWAYS TO DENSE STREETS

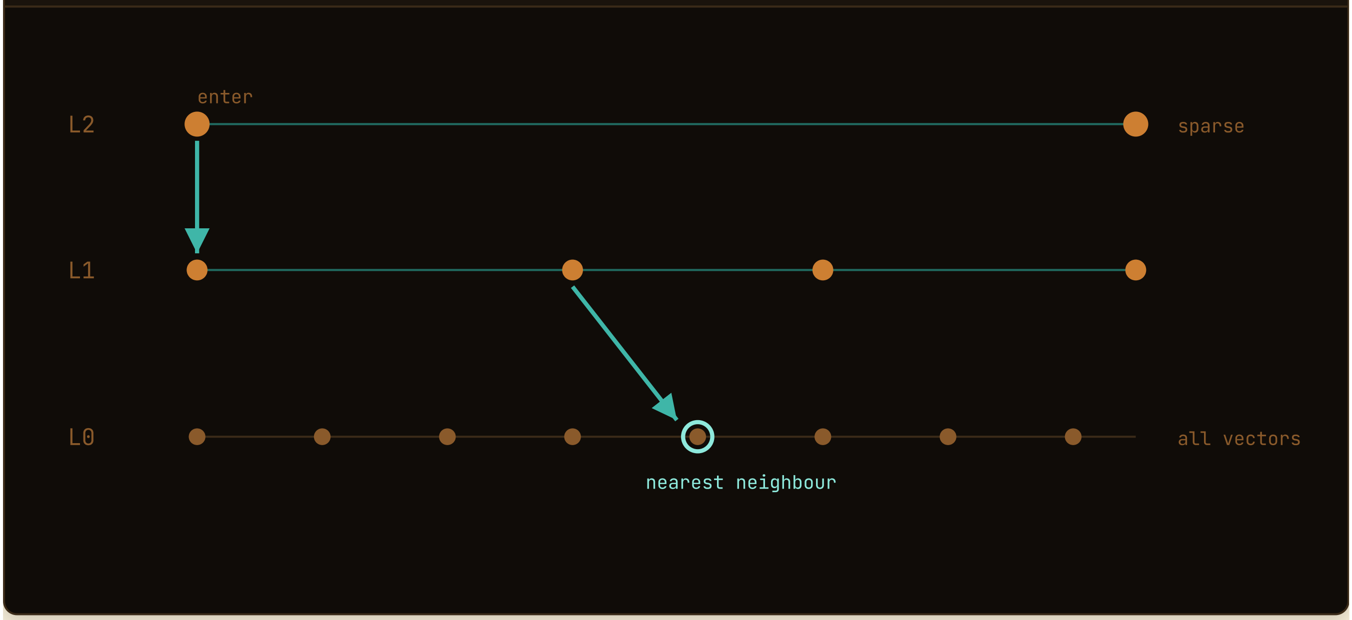

FIG. 7 — Big leaps up top, careful local search at the bottom — few comparisons, high recall.

⌥ UNDER THE HOOD — THE KNOBS

M (default 16): how many friends each node keeps — more friends, better recall, more memory. **efConstruction** (default 200): how hard it searches *while building*. **ef_search**: how hard it searches *at query time* — the size of the candidate "beam," and your live **recall ↔ speed dial** (you will see this exact knob in the benchmarks). Two refinements lift Quiver above a textbook version: (1) the **diversity heuristic** — when choosing friends, prefer ones pointing in *different directions*, so edges span the space instead of clumping, materially improving recall on clustered data; (2) **soft deletes** — a deleted point is tombstoned (kept as a navigation stepping-stone, never returned), and the search widens its beam to compensate, so recall holds even after many deletes.

Vamana / DiskANN — the graph built for *disk*

HNSW assumes the graph lives in RAM. **Vamana** (behind Microsoft's DiskANN) builds a single flat graph engineered so it can live on an **SSD** and still answer queries in very few disk reads — the key to Quiver's memory wedge. Its secret is the **RobustPrune α-slack rule**: when picking a node's neighbours it keeps the closest, then drops any candidate the chosen one is already α× closer to — forcing edges to span *long* distances, so greedy search reaches anywhere in very few hops (very few SSD page reads).

IVF — the "library card catalogue"

IVF (Inverted File) divides and conquers by neighbourhood: cluster all vectors into cells, and at query time scan only the few cells nearest the query.

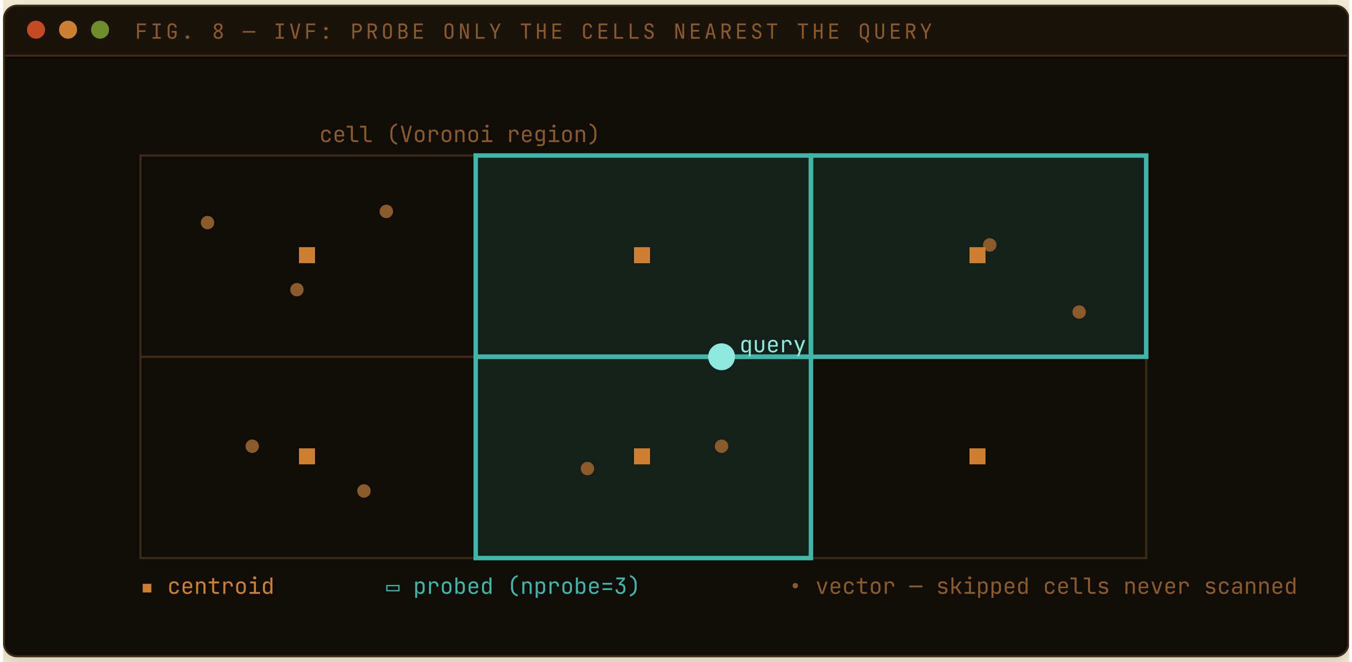

FIG. 8 — `nprobe` (how many cells to check) is IVF's recall ↔ speed dial.

Table 3 — choosing an index

INDEX	LIVES IN	STRENGTH	USE WHEN
HNSW	RAM	Highest QPS at high recall	You have the RAM and want raw speed
Vamana / DiskANN	SSD (compressed in RAM)	Tiny RAM footprint	Datasets bigger than your RAM budget
IVF	RAM or SSD	Fast build, predictable memory, easy updates	Streaming data, simpler tuning

4.3 Quantization: the memory-frugality superpower

One million 768-d vectors at 4 bytes each is **3 GB** of RAM just for the vectors; ten million is **31 GB** — it does not fit on a laptop. **Quantization** is lossy compression for vectors: shrink each into a tiny "code," search on the codes, accept slightly fuzzy distances.

Table 4 — Quiver's three quantizers

QUANTIZER	CODE SIZE	COMPRESSION	HOW IT WORKS
Scalar (SQ)	`dim bytes`	`~4×`	Each number stored as an 8-bit int, not a 32-bit float
Product (PQ)	**`m bytes`**	**`up to 32×`**	**Split into chunks; replace each with the ID of its nearest learned prototype**
Binary (BQ)	`dim/8 bytes`	`~32×`	Keep only the *sign* of each number (1 bit); compare by fast bit-counting

Product Quantization, simply. Imagine describing a face not with exact measurements but as "nose type #7, eyes type #3, jaw type #12." If everyone shares a catalogue of types, those three small numbers reconstruct the face approximately. PQ learns a *codebook* of prototype chunks, then stores each vector as a handful of prototype IDs — a 3072-byte vector becomes ~96 bytes, **32× smaller**. The genius that makes lossy compression safe:

> THE APPROXIMATE-THEN-RE-RANK PATTERN — THE ENGINE'S GOLDEN RULE
>
> **1. Rank cheaply on the tiny codes.** Use the compressed vectors to quickly find the top 100–800 candidates. Fast, but fuzzy.
>
> **2. Re-rank precisely on the originals.** Fetch the full-precision vectors for only that shortlist, compute exact distances, return the true top 10. You recover almost all the recall "lost" to compression.

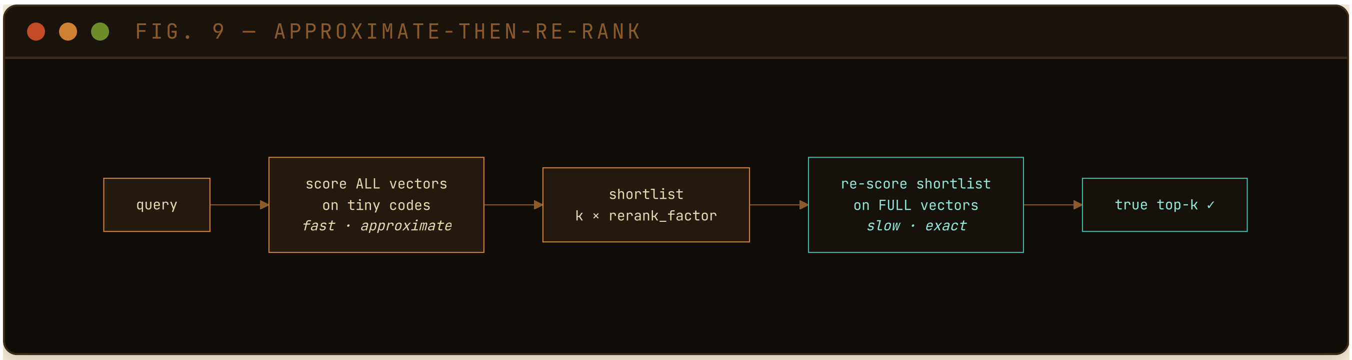

FIG. 9 — The shortlist depth (`rerank_factor`) is the master recall ↔ latency dial. A deeper pool can only *add* true matches.

⌥ UNDER THE HOOD

PQ uses **asymmetric distance computation (ADC)**: the query stays full-precision and a small lookup table is precomputed once per query, so scoring each code is a few table lookups and adds — extremely fast. Binary quantization leans on the SIMD **Hamming kernel** from §4.1: XOR the sign-bit codes, count differing bits, which a CPU does blisteringly fast — an ideal coarse pre-filter before the exact re-rank.

4.4 Putting it together: the disk-resident path *(the "32×" headline)*

Combine the **Vamana graph** (built for SSD) with **Product Quantization** (32× smaller), and you get Quiver's signature feature.

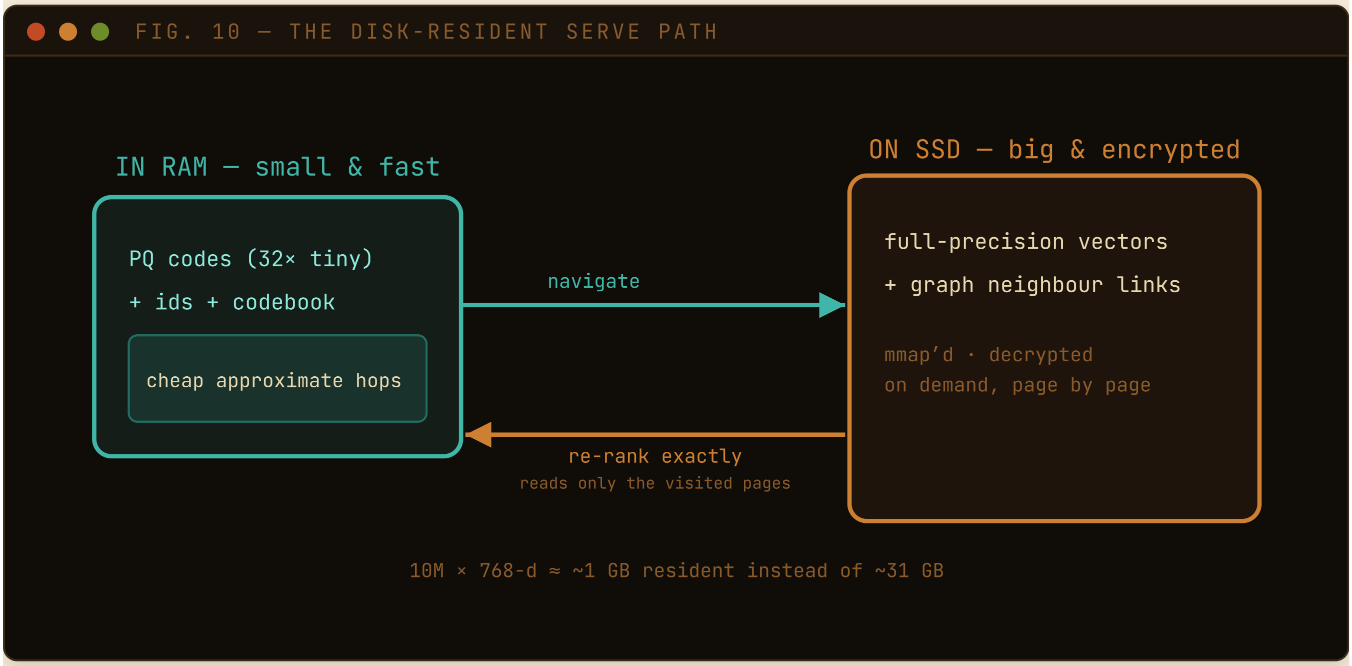

FIG. 10 — Only tiny PQ codes stay in RAM; full vectors and the graph live encrypted on SSD, touched only when visited.

The result, in Quiver's measured terms: a dataset serves from **roughly its PQ-code footprint** instead of the full vectors. For a 10M × 768-d collection that is about **~1 GB resident instead of ~31 GB** — exact arithmetic, and why "hundreds of millions of vectors on a laptop" is a real claim, not marketing.

4.5 Filtering: search by meaning *and* by rules

Real queries are hybrid: "films like Blade Runner, **but only sci-fi released after 1980**." That is a similarity search **plus** a structured filter on metadata. Quiver expresses filters as a **typed predicate tree** you can nest arbitrarily (`eq, ne, in, lt, lte, gt, gte, exists` plus `and/or/not`, over dot-paths like `user.age`):

FILTER — JSON WIRE FORM

```
{ "and": [
    { "eq":  { "field": "genre", "value": "sci-fi" } },
    { "gt":  { "field": "year",  "value": 1980 } },
    { "not": { "eq": { "field": "rating", "value": "G" } } }
] }
```

The interesting part is *how* it runs the filter — the planner picks one of two strategies:

STRATEGY	WHAT IT DOES	CHOSEN WHEN
Pre-filter	Find rows matching the filter first (via a secondary index), then search similarity only over those	The filter is **selective** (few rows) — e.g. `user_id = 42`
Post-filter	Search similarity first, then drop results that fail the filter	The filter is **broad** (many rows) — e.g. `year > 1980`

WHY BOTH

Pre-filtering a *broad* filter wastes time building a huge candidate set; post-filtering a *selective* filter risks all candidates being filtered out, leaving nothing. The filter is **re-checked on every surviving result**, so the answer is always exact — the planner only affects speed, never correctness.

4.6 Hybrid search: combining meaning with keywords (RRF)

Sometimes pure semantic search is not enough — someone searching a product code like "SKU-4417" wants that *exact* term, not "vibes." **Hybrid search** runs both a **dense** (embedding) search and a **sparse** (keyword/term-weight) search, then merges the two ranked lists. But their scores live on incomparable scales. The elegant fix is **Reciprocal Rank Fusion** (RRF): ignore scores, use only *rank*.

RECIPROCAL RANK FUSION

```
# each list gives a doc  1/(k0 + rank)  points; sum across lists
RRF(doc) = Σ  1 / (k0 + rank_in_list)      (k0 = 60, the standard constant)
         over each result list
```

Because it is purely rank-based, RRF needs **no score normalisation** — which is exactly why it is the robust, industry-standard fuser. A document ranking high in *both* lists wins. A sparse vector (e.g. from a SPLADE model) rides inside the point's payload, so enabling hybrid search needs *no change to the on-disk format*.

⌥ UNDER THE HOOD — MULTI-VECTOR / COLBERT

Quiver also supports **late-interaction (ColBERT-style)** retrieval: a document is stored as a *set* of token vectors and ranked by **MaxSim** (for each query token, find its best-matching document token, then sum). Cleverly, each document is modelled as a *group of ordinary rows* in the same storage engine — so there is no new on-disk format and the crash-safety guarantees are untouched. A ColBERT corpus (many small token vectors) is exactly the pool the IVF+PQ compression path was built for.

PART 5 DURABILITY

How It Survives `kill -9`

A database has one sacred promise: if it told you a write succeeded, that write is not lost — not to a crash, not to a power cut.

5.1 The Write-Ahead Log (WAL)

The idea is older than databases and simple: **write down what you are about to do *before* you do it.** Like a ship's logbook — before changing course, log the new heading. If interrupted, the log tells you exactly where things stood. In Quiver, every mutation is encoded, **appended** to the WAL, **fsync'd** (forced all the way to physical disk, not just the OS cache), and *only then* acknowledged. That ordering is the entire guarantee: once you get "ok," the record is physically on disk. The index update happens *after* the ack — and if the machine dies first, recovery replays the log to redo it.

5.2 Catching corruption: checksums and torn writes

What if a crash happens *mid-write*, leaving a half-written record? Or a disk bit silently rots? Every WAL record is framed with a length prefix and a **CRC32C checksum**:

ON-DISK FRAMES & PAGES

```
WAL frame:
  [ length:u32 ][ CRC32C:u32 ][ … the (encrypted) record bytes … ]
                    |
       recovery reads frames until one FAILS its length/CRC check
       — the signature of a crash mid-append — and treats
       everything after it as "never happened" (point-in-time recovery)

Data page (16 KiB):
  [ magic | version | type | page_id | lsn | payload_len | CRC32C ]   ← 32-byte header
  [ …data… | zero-padding to 16 KiB ]
```

FIG. 11 — Length + checksum framing turns a torn tail into a clean, detectable boundary.

On recovery, Quiver reads frames until one **fails its check** — the unmistakable signature of a crash mid-append — and treats everything from that point on as "never happened." Because the log is append-only and every record was fsync'd before its ack, the *only* place a broken frame can legitimately appear is the very tail. A torn trailing record was, by definition, never acknowledged, so discarding it loses nothing. The same discipline applies to the 16 KiB data pages: each carries its own CRC, so **corruption is detected on read and never silently served**.

THE BOTTOM LINE

Acknowledged writes survive `kill -9` and power loss; corruption is always *detected* rather than served as a wrong answer; and all of this holds **whether or not encryption is on**, because the checksums guard the plaintext path and encryption sits on top. Quiver's test suite literally kills the process with `kill -9` mid-operation and asserts the data is intact on restart — the "crash gate."

5.3 Many readers, one writer

Durability is about *not losing* writes; throughput is about *serving* reads. Quiver is **single-writer, many-reader**: the server guards the engine with a reader–writer lock (ADR-0057). A search takes the *shared* lock, so many searches run **in parallel** — read throughput scales with cores instead of serializing on one mutex. A write takes the *exclusive* lock; the single-writer model, and everything in §5.1–5.2, is unchanged.

There is one subtlety. Some writes (an HNSW in-place update, a bulk load) deliberately **defer** the index rebuild to batch the work. A reader that lands on a collection whose index is still stale gets a signal rather than a wrong answer: it takes the write lock *once* to rebuild the index (`ensure_indexed`), then serves — and every reader after it is concurrent again until the next write. Critically, this moves read *visibility*, never *durability*: a query may not yet see a write that committed a millisecond ago, but it never sees a half-applied one, and the WAL-fsync acknowledgement above is untouched.

STAGED, NOT SKIPPED

The reader–writer lock lets reads run concurrently *with each other*; a write still briefly excludes them. The fully lock-free path — where reads proceed *during* writes by traversing an immutable, atomically-swapped per-collection snapshot (arc-swap + epoch reclamation) — is designed and staged as the successor in ADR-0057, to land once a measured read-concurrency ceiling on real hardware justifies the reclamation complexity. Quiver ships the honest step now rather than the risky leap.

PART 6 SECURITY

Security, the Defining Feature

This is where Quiver makes its strongest claim. We go from the disk outward.

6.1 Encryption at rest — *on by default*

Most databases make you opt *in* to encryption. Quiver makes you opt *out* — and won't let you, on a non-loopback bind. Out of the box the server demands a 256-bit master key and **seals every durable byte** — segments, the manifest, *and* the write-ahead log — with **XChaCha20-Poly1305**, a modern, audited authenticated cipher. "Authenticated" (AEAD) matters: it not only hides data, it **detects tampering** — flip a single bit and decryption fails loudly instead of returning subtly wrong data.

> ⌥ UNDER THE HOOD
>
> Only audited cryptography is used — RustCrypto's AEAD/KDF and `rustls` (backed by the audited `ring` library) for TLS. **No home-grown primitives, ever** — the cardinal rule of applied cryptography. Key material is wrapped in zeroizing types so it is scrubbed from memory when dropped.

6.2 Envelope encryption + crypto-shredding

Instead of encrypting everything with the one master key, Quiver uses a **two-level key hierarchy**. Each collection gets its own random **Data-Encryption Key (DEK)**, itself stored encrypted ("wrapped") under the master key.

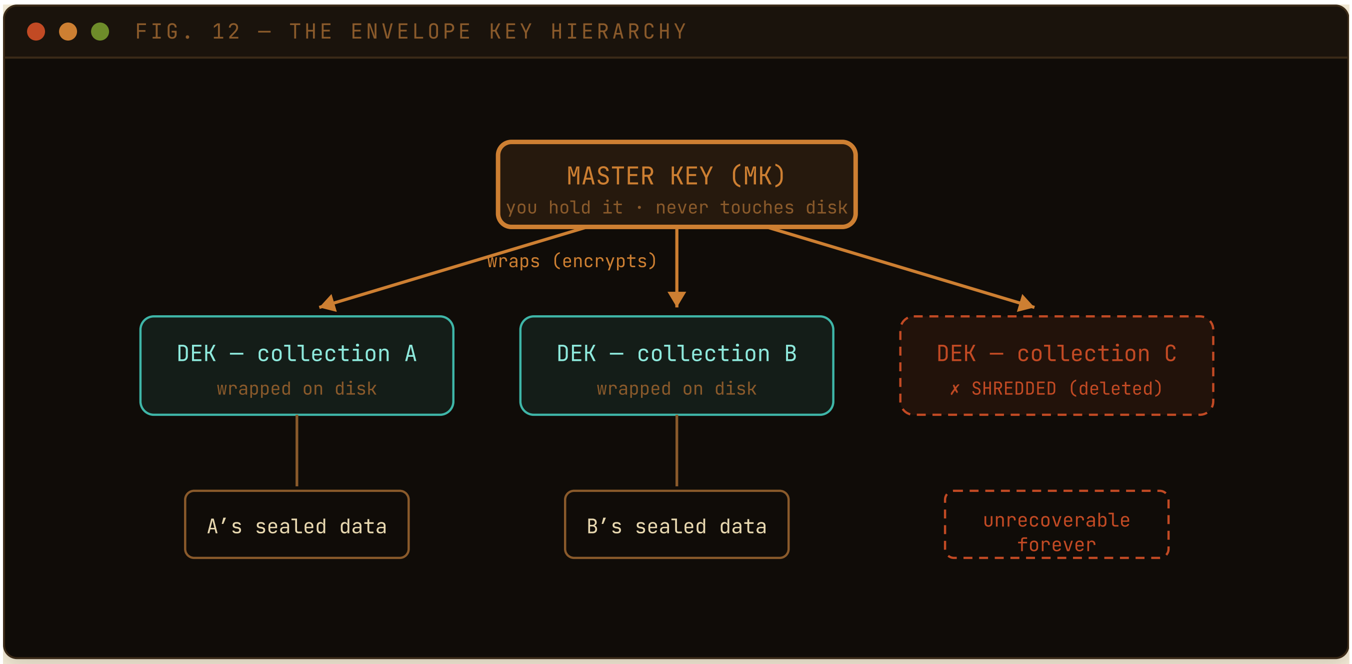

FIG. 12 — Per-collection keys, each wrapped by the master key — the structure that makes instant, provable deletion possible.

> CRYPTO-SHREDDING — INSTANT, PROVABLE DELETION
>
> To permanently delete a collection, Quiver does not overwrite gigabytes. It just **deletes that collection's tiny wrapped DEK file.** The DEK existed nowhere else. Once gone, the collection's encrypted bytes are **mathematically unrecoverable — even by the holder of the master key, even from a backup tape that still has the ciphertext.**

This is the gold-standard pattern for the GDPR "right to erasure": you can *prove* deletion happened (the key is provably gone) without trusting that every copy on every disk and backup got physically scrubbed. Quiver's tests demonstrate exactly this — seal a page, shred the collection, then show a fresh key-ring with the correct master key can no longer decrypt it.

6.3 In transit and access control

- **TLS / mTLS:** encrypted connections required for any non-loopback bind; optional mutual TLS makes clients prove identity with a certificate too.
- **Default-deny RBAC:** access by scoped API key, with a **role** (`read ⊆ write ⊆ admin`) and a **collection scope** (exact names, or an `acme.*` prefix for per-tenant isolation). Over-reach returns `403`; listing even *hides* out-of-scope collections.
- **Append-only audit log:** every mutating/admin operation and every denial is recorded — who, what, which resource, what outcome — but **never the secret**.
- **Query cost limits:** the server caps how expensive a single query can be, closing off "authenticated denial-of-service."

6.4 The frontier: encrypting the *vectors* themselves

By default, client-side encryption protects **payloads** (metadata): you can seal `{"ssn":"…"}` with a key the server never sees, while leaving `{"genre":"sci-fi"}` cleartext so the server can still filter on it. But can a server rank vectors it cannot read? Quiver offers two honest, opt-in answers — and is scrupulous about the trade-offs, because **no scheme gives fast server-side ranking, zero leakage, and good performance all at once.**

Table 5 — the two encrypted-vector modes

MODE	SERVER SEES	SERVER RANKS?	SECURITY	HONEST COST
client_side	Only opaque ciphertext + a zero placeholder	No — learns *nothing*	Strongest (IND-CPA)	Client fetches the (pre-filtered) set and ranks locally
dcpe (experimental)	Ciphertext it can compare by approximate L2	Yes — without the plaintext or key	Weaker — not semantically secure	Leaks the approximate distance-ordering *by design*; L2-only

The DCPE mode implements a *published* academic scheme ("Scale-And-Perturb" distance-comparison-preserving encryption) built only from audited primitives, with the paper's hardening steps; the docs tell you to read the threat model before using it.

> THE SECURITY-FIRST ETHOS IN MINIATURE
>
> A lesser project would ship DCPE and call it "encrypted search," full stop. Quiver ships it behind an experimental flag, names the exact academic paper, and spells out precisely what it leaks and which attacker breaks it. That candour is the point.

Benchmarking Quiver — Method, Results, Verdict

Performance claims are worth only the methodology behind them. Here is how we measured, what we found, and what it proves about Quiver's value.

7.1 Why we benchmark at a fixed recall

It is trivial to make a vector search "fast" — just return less-accurate results. So a throughput number on its own is meaningless. Every figure here is reported **at a fixed recall bar** (recall@10 ≥ 0.95 unless noted): the systems are tuned until they all find ~the same fraction of the true nearest neighbours, and only *then* is their speed compared. That is the only honest way to compare ANN engines.

7.2 The method

- **Same box, every system.** An `ann-benchmarks`-style harness runs each database on one machine — an Intel i7-12700H laptop, 20 threads, 15.5 GB RAM — so the comparison is apples-to-apples.
- **Seven competitors:** FAISS, Qdrant, Milvus, Chroma, pgvector, LanceDB, Weaviate. Milvus is run as the full *server* (Docker), not the in-process Lite build.
- **Honest caveats, stated up front:** FAISS/Chroma/LanceDB run *in-process*, so their reported RAM is inflated by the Python harness (not directly comparable to the isolated-server numbers); Qdrant memory-maps vectors to disk by default (so its RAM looks tiny); and Quiver's "build" time is the *bulk-ingest* path (`points:bulk`) measured as honest *time-until-queryable* — ingest plus the deferred index build forced by the first query, the same thing every competitor's build column captures.
- **Never fabricate.** Absolute RSS on reference hardware and the 10M-scale disk path are explicitly marked "pending" rather than guessed.

7.3 Quiver's own recall ↔ speed curve (SIFT1M, 1M × 128-d, in-memory HNSW)

This single table shows the central trade-off of the whole field: as you turn the `ef_search` dial up, recall climbs and throughput falls.

Table 6 — Quiver SIFT1M, HNSW (M=16, efC=200), single thread

EF_SEARCH	16	32	64	128	256
recall@10	0.793	0.895	0.958	0.986	0.995
QPS (1T)	1539	1424	1222	955	701
p95 latency (ms)	0.8	0.8	1.0	1.3	1.7

FIG. 13 — THE RECALL ↔ THROUGHPUT CURVE

FIG. 13 — RAG pipeline that re-ranks anyway? Run at ef=64. Legal discovery where a miss is unacceptable? ef=256.

7.4 Head-to-head (SIFT1M, peak single-thread QPS at recall@10 ≥ 0.95)

Table 7 — every system on the same box; [1] in-process RSS inflated · [3] Qdrant disk-backed

SYSTEM	RECALL@10	QPS (1T)	P95 (MS)	RSS (MB)	BUILD
FAISS 1.14	0.968	3842	0.4	1234[1]	82 s
Quiver v0.20	**0.958**	**1222**	**1.0**	**2069**	**581 s[2]**
Chroma 1.5	0.977	1009	1.1	3752[1]	153 s
Weaviate 1.27	0.983	663	1.7	2218	38 min
Milvus 2.5 (server)	0.986	649	1.9	2075	26 s
Qdrant 1.13	0.974	358	4.5	258[3]	98 s
pgvector 0.7	0.980	118	11.8	1291	132 s
LanceDB 0.33	0.557[4]	219	5.3	2475[1]	15 s

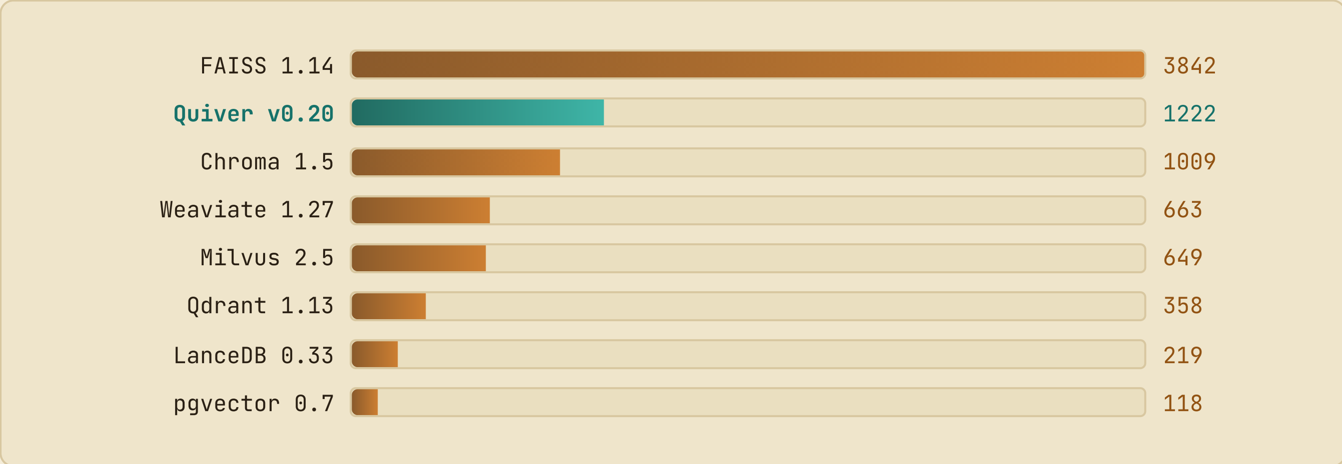

FIG. 14 — Single-thread QPS at recall@10 ≥ 0.95 (v0.20.0). Quiver is **second only to FAISS**, with the field's second-best tail latency (1.0 ms).

7.5 The harder test: GIST1M (1M × 960-d)

GIST1M is higher-dimensional and far more demanding — most systems plateau below recall 0.95 in a bounded sweep. Here Quiver **matches FAISS on recall (0.923 vs 0.919)**, and on the v0.20.0 engine is markedly faster than v0.18.0 at the same recall (182 → 268 QPS, p95 7.7 → 4.4 ms).

Table 8 — GIST1M, each system at its most efficient config (best point at ef_search ≤ 256)

SYSTEM	RECALL@10	QPS (1T)	P95 (MS)	RSS (MB)
Quiver v0.20	**0.923**	**268**	**4.4**	**10117**
FAISS 1.14	0.919	471	2.7	7526[1]
Chroma 1.5	0.790	577	2.1	8156[1]
Weaviate 1.27	0.828	418	2.8	8880
Qdrant 1.13	0.955	185	6.3	391[3]
Milvus 2.5 (server)	0.961	53	29.4	6821
pgvector 0.7	0.980	8	194	4393

The three systems that clear 0.95 here (Qdrant, Milvus, pgvector) pay for it in latency/QPS. LanceDB did not complete GIST1M — building a 960-d/1M index in-process exhausted memory (an honest DNF even with 27 GB of swap, not a fabricated row).

7.6 The real wedge: memory at a fixed recall

Critically — and Quiver says this itself — the tables above are an **in-memory HNSW** comparison, so they are *not* where Quiver's memory wedge shows up. The wedge is the **disk-resident path** (Part 4.4), which holds only PQ codes in RAM.

~32×	~1 GB	~31 GB	1.000
LESS RAM AT FIXED RECALL	RESIDENT · 10M × 768-D	FULL-PRECISION, IN RAM	DISK-PATH RECALL@10 (SIFTSMALL)

FIG. 15 — On SIFTSMALL the disk path serves recall@10 up to 1.000 while holding only PQ codes in RAM. The head-to-head RSS vs Qdrant/LanceDB is explicitly reference-hardware-pending — never fabricated.

7.7 The verdict — the value of Quiver

WHAT THE NUMBERS PROVE

On a like-for-like in-memory test, Quiver is **second only to FAISS** in throughput-at-recall and second-best in tail latency — ahead of Chroma, Weaviate, Milvus, Qdrant, pgvector, and LanceDB — and on the punishing 960-d GIST1M it **matches FAISS on recall.** For a from-scratch engine, that is genuinely competitive territory.

But raw QPS was never the point. Quiver's value is the **combination** the benchmarks bracket:

- **Competitive speed & recall** — you are not trading performance away to get the other benefits. It holds its own against purpose-built C++ engines.
- **The memory wedge** — the differentiator the head-to-head table deliberately does *not* capture: ~32× less RAM at a fixed recall via the disk-resident PQ path, turning a "needs a big server" dataset into a "runs on a laptop" one.
- **Security by default** — encryption-at-rest on, crypto-shredding, optional encrypted vectors, audited crypto only. No competitor in this table makes that its default posture.
- **One binary, two modes, agent-native** — embeddable *and* server, with SDKs, a cockpit, and an MCP server, from a single static binary.
- **Trustworthy numbers** — every caveat is stated; absolute figures pending reference hardware are marked, not guessed. You can reproduce the comparative standings yourself.

Put plainly: **if you want competitive ANN performance with security on by default and a memory footprint small enough to self-host on modest hardware, Quiver occupies a niche the incumbents leave open.**

The Decisions Behind the Code

Great engineering is visible in the choices — especially the restraint. Each is recorded as an Architecture Decision Record (ADR) in the repo.

Table 9 — the decisions that define Quiver

DECISION	WHAT THEY CHOSE	WHY
Language	Rust	Memory safety without a garbage collector → predictable low latency and no whole classes of security bugs.
Storage engine	Built from scratch (no RocksDB/SQLite/LMDB)	Full control over the on-disk format, encryption, and crash semantics — the core differentiators.
Cryptography	Only audited libraries (RustCrypto, ring/rustls)	"Don't roll your own crypto" — the one universal rule of security engineering.
Crash safety	WAL + fsync-before-ack + CRC + point-in-time recovery	The non-negotiable database promise, proven by a `kill -9` test gate.
Scope	Single-node excellence first; distribution is a labelled stretch	Do one thing superbly before doing everything adequately.
Honesty	No number ships unless reproducible on documented hardware	Trust is the product when you store someone's data.
Error handling	`unwrap()` / `expect()` banned by lint	Force every failure to be handled — no surprise panics in production.

THE THROUGHLINE

Correctness and security first, performance second, features last — and never lie about any of them.

PART 9 BATTERIES

Beyond Search: RAG, Full-Text & Operations

A search engine is only half a product. The rest is the work of using it *— turning text into vectors, mixing keyword and meaning, backing data up, watching the system breathe, and keeping one noisy client from starving the others.*

9.1 Bring your own *nothing*: server-side embedding

The number-one friction in retrieval is the embedding step: "I have text, but the database wants vectors, so I must run a model myself." Quiver stays model-agnostic at its core — but at the *edge* (never in the engine) it offers an opt-in, per-collection embedding hook. Configure a provider, then send **text** and let the server embed it.

> THE CORE IDEA
>
> `upsert_text` sends a document's words; the server embeds them with the collection's configured provider and stores the vector. `search_text` embeds your query the same way, searches, and can **rerank** the shortlist with a second, sharper model — retrieve-then-rerank in a single call.

The provider is chosen by configuration, never hard-wired to one vendor. Secrets are never stored: a config field names an *environment variable*, resolved at startup so a missing key fails fast.

Table 9 — built-in embedding / rerank providers (opt-in, per collection)

PROVIDER	SHAPE	NOTES
OpenAI	`/v1/embeddings`	The de-facto standard shape; also used by many local servers.
Ollama	OpenAI-compatible	Local models on your own hardware — no data leaves the box.
HTTP	OpenAI-compatible	Any endpoint that speaks the shape, parameterised by base URL + auth.
Cohere	`/v2/embed`, `/v2/rerank`	Its own shape, including a native reranker.
`fake`	deterministic	A hash-to-vector stub for tests and offline demos — no network.

FIG. 16 — THE EMBEDDING & RERANK PIPELINE (SEARCH_TEXT)

1	`Client → Server`	`search_text("a fast fox", rerank: true)` — just words, no vector
2	`Server → Embedder`	embed the query with the collection's provider (key from an env var)
3	`Server → Engine`	HYBRID dense ANN ⊕ BM25 full-text, fused with RRF → an over-fetched candidate pool
4	`Server → Reranker`	SHARPER, SLOWER ✓ a cross-encoder scores each *(query, document)* pair and reorders the pool
5	`Server → Client`	the reranked top-k — one round trip, no model on the client

FIG. 16 — Embedding and reranking live at the server edge, opt-in and provider-agnostic; the engine still only ever sees vectors.

9.2 Full-text without leaving home: BM25

Meaning search is powerful, but sometimes you want the *exact word* — a product code, a name, an acronym a model has never seen. Quiver indexes text for classic keyword ranking too, with no extra dependency.

> TWO KINDS OF RELEVANCE, FUSED
>
> **Dense** (vector) search finds things that *mean* the same. **BM25** — the lexical workhorse behind Lucene/Elasticsearch — scores by *which rare words two texts share, and how often*. Quiver tokenises text (Unicode split, lowercase, stop-words, a **Snowball/Porter2 stemmer** so "running", "ran", and "runs" conflate), builds an inverted index, and scores with Okapi BM25. A single `query_text` can run dense, lexical, or — best of all — both, fused with RRF.

9.3 Backups that don't stop the world

Your data directory is already portable — stop the process and copy it. But you should not have to stop. An **online snapshot** captures a consistent copy of a *running* database, anchored at one point in its history.

FIG. 17 — THE ONLINE SNAPSHOT (CONSISTENT BACKUP)

1	`Admin → Server`	`POST /v1/snapshot { destination }`
2	`Engine`	take the single writer lock — no mutation is in flight
3	`Engine → Disk`	CONSISTENCY ANCHOR ✓ **checkpoint**: seal the in-memory buffer into segments and advance the WAL floor to the head
4	`Engine → Destination`	byte-copy the whole data directory — layout-independent, so it can never drift out of sync with the file format
5	`Restore`	open the copy under its keyring — it replays an empty WAL tail and is identical to the source at snapshot time

FIG. 17 — Consistency comes from the checkpoint plus the single-writer lock; the copy itself never parses a segment, so it captures new artifacts for free. The crash gate is untouched.

9.4 Seeing inside: metrics & tracing

An operator needs to see the system breathe. Quiver serves a Prometheus `/metrics` endpoint (open, so a scraper needs no key) with real request counters and latency histograms per route, plus security counters; engine operations carry secret-free tracing spans, and an importable Grafana dashboard ships in the repo. The metrics below are also the vocabulary the rest of this guide measures in.

Table 10 — the metrics that matter, defined (with a worked example)

METRIC	IN PLAIN WORDS	WORKED EXAMPLE
recall@k	Of the *true* k nearest neighbours, what fraction did the (approximate) search actually return?	Ask for 10; 9 of the true top-10 come back → recall@10 = **0.90**.
QPS	Queries served per second — raw throughput.	30,000 searches in 20 s → **1,500 QPS**.
p50 / p95 / p99	Tail latency: the time under which 50 / 95 / 99 % of requests finish. p99 is what your slowest users feel.	p95 = 8 ms means 95 of 100 queries answered within 8 ms.
RRF	Reciprocal Rank Fusion — merge two ranked lists using only ranks: score = $\Sigma\ 1/(k_0+\text{rank})$.	Rank 1 on the dense list + rank 3 on BM25 (k_0=60) → 1/61 + 1/63.
IDF	Inverse Document Frequency — rare words carry more signal than common ones; the heart of BM25.	"the" appears everywhere → ~0 weight; "xylophone" is rare → high weight.
RSS	Resident Set Size — the physical RAM a process actually holds. The memory-frugality headline.	Disk path: ~1 GB resident for 10 M vectors vs ~31 GB fully in RAM.
build time	Wall-clock time to ingest a dataset and build its index — the "time to first query."	One bulk load: one WAL fsync + one index pass, not N round trips.

9.5 Fairness under load: per-key rate limiting

THE TOKEN BUCKET

Each API key gets a bucket that refills at a steady rate up to a burst capacity; every request spends one token. Spend faster than the refill and the bucket empties — further requests get `429 Too Many Requests` with a `Retry-After` and the standard `RateLimit-*` headers. It is opt-in (off by default), enforced at one auth choke point so *every* REST and gRPC call is covered, and per-node in memory — a clustered limiter is a job for the distributed-mode design.

9.6 The client fleet

One engine, many front doors. Beyond REST and gRPC, Quiver ships SDKs for **Python** (sync + async), **TypeScript**, and **Go** (standard-library only), each mirroring the same surface — collections, points, search, hybrid, full-text, server-side embedding, and snapshots. The three are kept at **method parity**: alongside the core calls, each carries the same bulk/maintenance helpers — batched upload (`upsertIter` / `UpsertBatch`), a `scroll` over a whole collection for export and re-embedding, and a paged `deleteByFilter` for GDPR erasure — and the TypeScript client takes a sync *or* async iterable, the Go client a context on every call. For AI agents, an **MCP server** exposes the database as tools (create, upsert, search, hybrid, `database_stats`, `delete_collection`, `snapshot`), so an autonomous agent can not only query Quiver but *operate* it.

9.7 The roads not (yet) taken

Restraint is a feature. Two large capabilities are *designed* — written up as architecture records — but deliberately unbuilt until the single-node story is unbeatable and a real need appears: **distributed / sharded mode** (hash sharding + scatter-gather + per-shard Raft for write HA) and **GPU acceleration** (behind the index trait, CPU-default). A third — **concurrent reads** — has taken its first real step: reads now run in parallel behind a reader–writer lock (§5.3), with the fully **lock-free MVCC** path (versioned arc-swap snapshots so reads never wait on the writer at all) staged as the successor. Each ships as a design ADR first; none compromises the single static binary.

A REFERENCE

Glossary for the Newcomer

Every term this guide leans on, defined once, in plain language.

`Vector / Embedding`
A list of numbers representing the *meaning* of content. Similar meanings → nearby numbers.

`Embedding model`
The neural network that turns text/images into vectors. (You bring your own.)

`Metric`
How "distance" is measured: cosine (angle), L2 (ruler), dot product (angle + length).

`ANN`
Approximate Nearest Neighbour — finding the *almost-* perfect closest vectors fast.

`Recall`
Accuracy of approximate search: of the true top-K, how many were found. (1.0 = perfect.)

`k / top-k`
How many results you want ("the 10 most similar movies").

`Index`
The data structure that makes search fast (HNSW, Vamana/DiskANN, IVF).

`HNSW`
A layered "navigable graph" index; fast, lives in RAM.

`Vamana / DiskANN`
A graph index engineered to live on SSD with a tiny RAM footprint.

`IVF`
An index that clusters vectors into cells and searches only the relevant ones.

`Quantization`
Lossy compression of vectors into tiny codes (Scalar ~4×, Product/Binary ~32×).

`Re-ranking`
After a fast fuzzy pass, recomputing exact distances on the shortlist.

`WAL`
Write-Ahead Log — the "write it down before doing it" log that survives crashes.

`fsync`
The command that forces data all the way onto physical disk.

`AEAD`
Encryption (e.g. XChaCha20-Poly1305) that also detects tampering.

`Crypto-shredding`
Deleting data permanently by destroying its (tiny) key.

`RBAC`
Role-Based Access Control: who may do what, to which data.

`RRF`
Reciprocal Rank Fusion — merging two ranked lists using only their ranks.

`Payload / metadata`
Structured data attached to a vector (`{"year":1982}`), used for filtering.

`MCP`
Model Context Protocol — lets AI agents call tools; Quiver exposes itself as such.

`BM25`
The classic keyword-ranking formula (Lucene/Elasticsearch): scores by which *rare* shared words two texts have, and how often.

`IDF`
Inverse Document Frequency — how rare (and therefore informative) a word is across the corpus.

`Stemmer (Snowball/Porter2)`
Reduces words to a root so “running”, “runs”, and “ran” match as one.

`Rerank`
A second, sharper pass (a cross-encoder) that reorders a retrieved shortlist for precision.

`Embedding provider`
An external model (OpenAI/Cohere/Ollama/HTTP) Quiver can call, at the edge, to turn your text into vectors.

`Snapshot / checkpoint`
An online, consistent backup of the whole database; the checkpoint seals memory to disk so the copy needs no replay.

`Token bucket`
A rate limiter: a bucket refills at a fixed rate; each request spends a token; empty → `429`.

`Prometheus / OpenTelemetry`
Standard formats for metrics (scraped from `/metrics`) and traces (spans) — what dashboards and alerting consume.

`recall@k`
The fraction of the *true* k nearest neighbours an approximate search actually returns — search quality.

`p95 / p99`
Tail latency — the time under which 95 % / 99 % of requests complete; what slow users feel.

`RSS`
Resident Set Size — the physical RAM a process actually uses; Quiver’s memory-frugality headline.

B SOURCES

References & Repository Documentation

Every claim in this guide traces to source code or to a documented, reproducible benchmark. The primary sources live in the Quiver repository.

Architecture & overview

R1 **Architecture overview & crate map**
`docs/architecture/overview.md`

R2 **C4 container view** · C4 system context
`docs/architecture/c4-container.md · c4-context.md`

R3 **Project README** — positioning, quickstart, benchmark tables
`README.md`

Indexes, kernels & quantization

R4 **HNSW** (Malkov & Yashunin, 2020; arXiv:1603.09320)
`crates/quiver-index/src/hnsw.rs · ADR-0007`

R5 **Vamana / DiskANN** (Subramanya et al., NeurIPS 2019)
`crates/quiver-index/src/vamana.rs · disk.rs · ADR-0019`

R6 **IVF** coarse Voronoi partitioning · SpFresh/LIRE updates
`crates/quiver-index/src/ivf.rs · ADR-0007 · ADR-0023`

R7 **Quantization** — scalar / product / binary
`crates/quiver-index/src/quant/ · ADR-0008`

R8 **SIMD distance kernels** — runtime dispatch + scalar fallback
`crates/quiver-simd/src/ · ADR-0009`

R9 **Hybrid (dense + sparse) search · RRF fusion**
`crates/quiver-query/src/sparse.rs · ADR-0043`

R10 **Multi-vector / late-interaction (ColBERT, MaxSim)**
`crates/quiver-index/src/colbert.rs · ADR-0028 · ADR-0034`

Storage, durability & security

R11 **Write-ahead log & page format** — CRC, recovery
`crates/quiver-core/src/wal.rs · page.rs · ADR-0004 · ADR-0005`

R12 **Envelope encryption & crypto-shredding**
`crates/quiver-crypto/src/envelope.rs · docs/security/crypto.md · ADR-0010`

R13 **Encrypted vectors** — client-side & DCPE
`docs/security/dcpe.md · client-side-vectors.md · ADR-0031 · ADR-0032`

R14 **Auth, RBAC, tenancy & threat model**
`docs/security/threat-model.md · ADR-0011 · ADR-0013`

Benchmarks & method

R15 **Head-to-head comparison (v0.20.0)** — SIFT1M + GIST1M, 7 competitors
`docs/benchmarks/results/comparison-v0.20.0/`

R16 **Benchmark methodology** · disk-path memory results · quantization trade-offs
`docs/benchmarks/methodology.md · results/disk-path.md`

R17 **The benchmark harness**
`bench/`

Decisions & project

R18 **Architecture Decision Records** (ADR-0001 … ADR-0044)
`docs/adr/`

R19 **Documentation site** (mdBook) — concepts → API → security → architecture
`apps/docs/ · build with` `just docs`

R20 **Security policy** · roadmap · contributing
`SECURITY.md · docs/roadmap.md · CONTRIBUTING.md`

Repository — **github.com/achref-soua/quiver** · License: AGPL-3.0-only · Edition v0.21.0 · All paths are relative to the repository root.

"A quiver holds arrows, and an arrow is a vector.
In mathematics a quiver is a directed graph —
which is exactly what an HNSW or Vamana index is."

```
quiver@cockpit:~$ ./read --status=complete ✓
```

The security-first vector database
Client-side-encryptable · memory-frugal · runs on a laptop

github.com/achref-soua/quiver
Open-source · AGPL-3.0-only